

Experiment 8 – Detect Hidden Data in Images Using StegExpose

Done by : 99240041247

Aim: To detect hidden (steganographic) data in image files using the StegExpose tool and analyze its detection accuracy.

Objectives:

1. Understand the concept of Steganography and Steganalysis.
2. Learn to use StegExpose for detecting hidden data.
3. Evaluate the accuracy of detection using different image datasets.

About StegExpose: StegExpose is an open-source steganalysis tool used to detect hidden information within image files. It utilizes multiple statistical detection techniques to identify inconsistencies caused by steganographic embedding .

Detection Techniques Used:

#	Technique	Description
1	Sample Pair Analysis (SPA)	Detects hidden data based on pixel pair statistics.
2	RS Analysis	Analyzes correlations between pixel groups to identify anomalies.
3	Chi-Square Attack	Examines irregularities in pixel frequency distributions.
4	Primary Sets	Detects embedding noise through variations in statistical distributions.

Requirements:

#	Component	Details
1	Operating System	Windows or Linux
2	Software Required	Java (JDK 8 or higher)
3	Tool	StegExpose.jar
4	Input Files	Normal and steganographically modified image files (.jpg, .png)

Algorithm / Procedure:

Step 1: Download and Setup

- Download the StegExpose.jar file from GitHub.
- Verify that Java JDK or JRE is installed and configured on your system.
- Place the StegExpose.jar file in a working directory for analysis.

ChSquare.class	moving from ide to text editor development	10 years ago	☆ 226 stars
ChSquare.java	moving from ide to text editor development	10 years ago	👁 19 watching
Fuse.class	moving from ide to text editor development	10 years ago	🍴 59 forks
Fuse.java	moving from ide to text editor development	10 years ago	Report repository
ImageFileManager.class	moving from ide to text editor development	10 years ago	Releases
ImageFileManager.java	moving from ide to text editor development	10 years ago	No releases published
PixelBenchmark.class	moving from ide to text editor development	10 years ago	Packages
PixelBenchmark.java	moving from ide to text editor development	10 years ago	No packages published
PrimarySets.class	moving from ide to text editor development	10 years ago	Contributors 2
PrimarySets.java	moving from ide to text editor development	10 years ago	b3dk7 enoog
README.md	fix simple typo	7 years ago	ipmi-13 Adam Leskis
README.md~	readme fix	10 years ago	Languages
RSAnalysis.class	moving from ide to text editor development	10 years ago	Java 100.0%
RSAnalysis.java	moving from ide to text editor development	10 years ago	
RunStegExpose.class	moving from ide to text editor development	10 years ago	
RunStegExpose.java	moving from ide to text editor development	10 years ago	
SamplePairs.class	moving from ide to text editor development	10 years ago	
SamplePairs.java	moving from ide to text editor development	10 years ago	
StegExpose.analysis.odt	adding dissertation	11 years ago	
StegExpose.jar	update report headers	11 years ago	
commons-math3-3.1.1.jar	moving from ide to text editor development	10 years ago	
dissertation.pdf	adding dissertation	11 years ago	
manifest.mf	moving from ide to text editor development	10 years ago	
roc.png	adding to readme	11 years ago	

Step 2: Prepare Image Files

Prepare two sets of image samples for analysis:

1. Original Images – Images that do not contain any hidden data.
2. Stego Images – Images in which data has been embedded using tools such as:
 - OpenStego
 - SilentEye
 - QuickStego
 - Suite Picsel Security

These two categories will be used to compare normal and steganographically modified images during detection.

- [OpenStego](#)
- [SilentEye](#)

Step 3: Run StegExpose

1. Open Command Prompt or Terminal.
2. Navigate to the directory that contains the StegExpose.jar file and the image samples.
3. Execute the following command to analyze the images:

Example command: `java -jar StegExpose.jar "C:\Forensics_Lab\Images"`

This command scans all images in the specified folder and generates a report indicating the likelihood of hidden data in each file.

```
java -jar StegExpose.jar <image_folder_path> -t 0.2 -a
```

Parameters:

#	Parameter	Meaning
1	-t	Folder path containing test images
2	0.2	Threshold value (default = 0.2)
3	-a	Analyze all images automatically

Step 4: Observe the Output

After the command is executed, StegExpose generates a report containing the analysis results.

Output File: StegExposeResults.csv Contents include:

- File Name
- Suspiciousness Score
- Classification (Clean / Stego)

Step 5: Analyze and Interpret Results

#	Suspicion Score	Classification
1	$\leq$ Threshold (for example, 0.2)	Clean Image
2	$>$ Threshold (for example, 0.2)	Stego Image

Tip: The threshold value (-t) can be adjusted to control the balance between false positives and false negatives during detection.

Sample Command:

```
java -jar StegExpose.jar C:\Images -t 0.3 -a
```

This command scans all images in C:\Images with a threshold of 0.3 and performs a full automatic analysis.

Result:

After performing the analysis:

- The CSV report displayed each file's suspicion score.
- Images with higher scores were identified as steganographic.

#	File Name	Suspicion Score	Classification
1	image1.jpg	0.12	Clean
2	image2.png	0.35	Stego
3	image3.jpg	0.28	Stego

Conclusion:

- By using StegExpose, hidden data within image files can be accurately detected through the use of multiple steganalysis algorithms. The tool generates a quantitative score that helps distinguish between clean and altered images, making it an effective solution for digital forensic analysis and investigation.

